

IoT Drift: Catch Me If You Can

Marina Melkonyan

BS in Data Science
American University of Armenia
marina.melkonyan@edu.aua.am

Gurgen Hovakimyan

Adjunct Lecturer
American University of Armenia
ghovakimyan@aua.am

Abstract—Long term IoT sensor networks continuously collect data that powers modern intelligent systems. However, as time passes, environmental changes, hardware aging, lighting conditions, weather patterns, and other real world factors can gradually alter the data being produced. These hidden shifts in feature distributions, known as feature drift, often remain unnoticed while silently reducing the reliability of machine learning models. This capstone investigates hidden feature drift in continuous IoT imagery streams using three real world long term datasets: urban streetlight camera imagery, agricultural orchard monitoring from Raspberry Pi sensors, and pest monitoring sticky trap cameras. Together, these datasets span multiple years, diverse environments, and different sensor types, enabling robust cross domain evaluation. Unsupervised deep learning methods, including variational and convolutional autoencoders, were trained on stable baseline periods and evaluated on later target periods to identify drift through reconstruction loss and latent distribution changes label free data. To improve trustworthiness, the project combines multiple model architectures, custom drift metrics, statistical testing, and metadata based validation to distinguish genuine environmental drift from temporary anomalies or sensor artifacts. Results show that hidden drift can be detected accurately and often rapidly, while detection performance depends on both domain differences and environmental variability. Overall, this research contributes an end-to-end, practical drift monitoring pipeline that is interpretable, transferable across domains, and suitable for long-running real-world IoT deployments.

Index Terms—Internet of Things, Feature Drift Detection, Autoencoder, Reconstruction Loss, Latent Space, Image Streams, Seasonal Drift

I. INTRODUCTION

Internet-of-Things (IoT) systems operate by continuously collecting data from distributed sensors deployed in dynamic real-world environments. As these environments evolve, the characteristics of the generated data streams also change over time due to factors such as weather conditions, sensor aging, hardware recalibration, and shifts in usage patterns [16], [17]. As a result, IoT data streams rarely remain statistically stable. This evolving nature of the data leads to feature drift, a subtype of concept drift where patterns learned from historical data no longer accurately represent current observations. In such settings, machine learning models that assume fixed data distributions tend to lose predictive accuracy over time, which directly affects the reliability and trustworthiness of long-term IoT analytics. Consequently, identifying and managing concept drift has become a central challenge in deploying robust learning systems for continuous IoT data streams [1].

Previous research has extensively studied concept and feature drift detection in streaming data, proposing statistical, window-based, and weighting-based approaches to identify

distributional changes in sensor measurements and derived features [2], [3]. Recent studies in IoT contexts demonstrate the effectiveness of unsupervised drift detection methods to identify drift in sensor signals and image-based streams, but evaluations are often limited to detection performance on generic benchmarks or isolated sensor modalities [4]–[6], [10].

Despite substantial progress, there remains a gap in integrated approaches that monitor feature drift in high-dimensional IoT data streams using unsupervised methods [3], [4], [10], and validating the underlying causes of detected drift. Most existing studies focus on detection accuracy, while limited attention is given to understanding how drift can be minimized based on different environments and how detected drift events can be validated against physical or environmental changes. This project aims to address this gap by applying unsupervised drift detection techniques to different real-world IoT imagery datasets, to find suitable metrics to systematically analyze feature drift behavior, and introduce a validator mechanism to improve the reliability and interpretability of continuous IoT data stream analytics.

Research Questions

- 1) How does the mean time to detection (MTTD) of the validator change when applied to cross-domain data streams compared to its baseline performance in the source domain?
- 2) What is the sensitivity of unsupervised reconstruction loss as a trigger for drift adaptation in IoT imagery, and how does this sensitivity correlate with the degree of environmental variance across long-term data streams?

II. LITERATURE REVIEW

A 2024 systematic review covering more than 200 studies on online learning in evolving data streams [1] revealed an important gap between research advances and real-world deployment. Although many drift detection methods perform well in controlled academic settings, they are often evaluated on balanced, low-dimensional, and labeled datasets. In contrast, real IoT systems produce continuous streams of high-dimensional sensor data that are typically unlabeled and frequently imbalanced, making drift detection much more challenging in practice [1], [2]. The review specifically points to the need for **unsupervised** multivariate approaches that can handle complex sensor data such as imagery, which closely matches the focus of this research.

Further research exploring how different weighting strategies influence detector sensitivity and response time shows that methods based on fixed windows often struggle when the strength of drift changes over time. In response, adaptive windowing approaches such as ADWIN [24] expand or contract the comparison window in real time based on a statistical hypothesis test, allowing them to remain sensitive to sudden shifts while avoiding excessive false alarms during stable periods. The unsupervised extension ADWIN-U [25] takes this further by removing the requirement for labeled error rates entirely, making it directly applicable to unlabeled IoT imagery. Despite this fact, adaptive approaches remain more stable and responsive. These findings reinforce the design choices made in this study and highlight the need for more flexible drift detection methods. In response, this research uses adaptive kernel bandwidth selection through the median heuristic in the **STKA** metric, together with bootstrap-based confidence intervals, allowing drift to be assessed more reliably than with fixed thresholds alone. The theoretical grounding for kernel-based distribution comparison is provided by the MMD framework [26], which gives a non-parametric, distribution-free test that is particularly well suited to the high-dimensional latent spaces produced by autoencoders.

Recent research shows that **VAE** [28] brings a structural advantage for IoT drift detection beyond mere dimensionality reduction [6]: because the encoder is trained to map inputs onto a regularized Gaussian prior, samples from a shifted distribution produce latent codes that deviate measurably from that prior, generating elevated reconstruction error and measurable distribution changes simultaneously. Research on IoT sensor calibration demonstrated that a VAE trained in clean sensor readings can detect and quantify drift caused by hardware aging through the reconstruction loss even without any labeled degradation examples [6]. The core mechanism is motivated by what is used here; however, the approach is changed. The **novel** solution suggested in this research is based on a simple but yet powerful idea: train different **autoencoder architectures** on a reference periods, then watch how hard they struggles to reconstruct images from a later period. If reconstruction gets difficult, we deal with **drift**.

Going deeper in the field, anomaly detection research using VAEs for image streams [30], [31] has established that reconstruction error is a reliable unsupervised proxy for distributional novelty, provided the model has been trained exclusively on normal data and does not over-generalize. Video anomaly detection methods [32] have extended this idea to temporal sequences, showing that frame-level reconstruction scores can be aggregated into reliable stream-level alarms - a principle directly applicable to **IoT image streams** that arrive at regular intervals.

Additionally, drift in IoT sensor networks presents challenges that extend beyond what generic streaming benchmarks capture. One of the most practically focused studies in this space demonstrated analytics tailored to heterogeneous IoT sensor networks, combining statistical window tests with sensor-level metadata validation to distinguish environmental drift from sensor faults [5]. The key finding was that drift detected purely from data streams is frequently confounded by

transient sensor anomalies like power fluctuations, calibration resets, and that correlating detected events with maintenance logs and environmental metadata dramatically reduces the false positive rate. The **validator module** in this capstone operationalizes this principle by cross-referencing detected latent drift with brightness changes, GPS consistency, fault detection rates, and confidence score shifts.

Moreover, research on concept drift localization in IoT intrusion detection streams [7] introduced the idea of decomposing a detected drift event into contributions from individual feature subsets rather than producing a single global alarm. This enables targeted remediation by identifying where in a feature space drift is concentrated. It is also worth noting that similarity metrics across neural network representations [13] provide a useful theoretical grounding for why latent-space comparisons are meaningful: when two distributions of latent codes diverge, they are likely encoding genuinely different visual content rather than superficially different pixels. Finally, the **streetlight dataset** [11], the **pomegranate orchard time series** [14], and the **stink bug trap imagery** [15] are each assembled for real-world monitoring contexts, providing the kind of messy, domain-specific conditions, which is exactly what makes them useful here. Real sensors break, lighting changes, traps get replaced, and trees grow; a drift detection pipeline that cannot cope with that kind of noise is not much use in practice [1], [2].

Taken together, **three gaps** emerge from the literature that this work directly targets. Existing drift detectors are rarely tested across genuinely heterogeneous IoT deployments, leaving their cross-domain generalisability largely unknown [3], [4], [10]. Studies also rarely measure how quickly drift is caught or how **detection sensitivity shifts** with environmental variability, giving practitioners no principled basis for tuning thresholds in new settings [1], [2], [27]. Finally, detected drift signals almost never get cross-checked against physical context, making it hard to tell real distributional change apart from transient noise like lighting fluctuations or hardware faults [5], [7]. This research closes all three gaps: multiple autoencoder architectures are trained on three real-world datasets with detection speed quantified via MTTD; a systematic sensitivity analysis links reconstruction error to environmental variance metrics; and a validator module cross-references latent drift with sensor metadata to produce trustworthy, interpretable alerts - bridging the divide between controlled benchmarks and the messy reality of long-running IoT deployments.

III. METHODOLOGY

In this section, we walk through how the problem of detecting and quantifying feature drift in IoT sensor imagery is approached without ever needing a single labeled example of what drift actually looks like. The pipeline is designed to be end-to-end: starting from raw images and metadata, moving through feature extraction and model training, and ending with a validated drift score that operators can act on.

Figure 1 illustrates the complete drift monitoring pipeline. Raw images from each IoT camera are ingested and paired with their metadata catalogue, then passed through one of two

feature extraction strategies (RGB histograms or ResNet18 embeddings). The extracted features feed an autoencoder trained exclusively on reference-period data; once frozen, the model scores every incoming target-period image by its reconstruction error. A statistical test compares the two error distributions, the MTTD clock starts at the period boundary, and a validator module cross-checks any declared drift event against sensor metadata before issuing an operator alert.

A. Datasets and Study Design

Three real-world IoT camera datasets are used, each posing a different kind of challenge for drift detection.

The first and primary dataset comes from a network of fixed urban streetlight cameras that have been operating continuously for four years [11]. Each image arrives with rich metadata: the captured timestamp, GPS coordinates, hardware fault flags, a model confidence score, and a daytime/nighttime label. To study seasonal drift, winter images (January–March, Q1) are treated as the reference period and summer images (July–September, Q3) as the target. This pairing maximizes environmental contrast in lighting conditions, color temperature, and vegetation are dramatically different across these windows, while the cameras themselves remain physically unchanged, making it an ideal tested. A **daytime/nighttime split** is applied as a secondary robustness check to confirm that any detected drift is not merely an artefact of changing daylight hours. Urban camera networks of this kind are increasingly deployed for smart-city monitoring [16], making drift robustness a practical operational concern.

The second dataset comes from cameras monitoring individual pomegranate trees across two growing seasons [14]. Unlike the streetlight case, each camera watches a different physical tree, so drift must be evaluated per sensor rather than globally: pooling all cameras together would mix genuinely drifting sensors with stable ones and obscure the picture.

Finally, the third dataset is a single camera monitoring an insect sticky trap over one complete field season [15]. Here, the summer period (Q3) acts as the reference and autumn (Q4) as the target. This dataset presents a unique complication: the physical trap is replaced multiple times during the season, each replacement resetting the visual appearance of the monitored surface. A core methodological challenge is therefore to distinguish genuine **seasonal drift** from these sudden trap-change artefacts - a problem that mirrors the broader challenge of separating environmental drift from sensor events documented in the IoT sensor analytics literature [5], [7], [21].

B. Feature Extraction: Two Approaches and Why They Differ

For all three datasets the first step is constructing a structured catalogue that associates every image with its capture timestamp, sensor identity, and any available metadata. The catalogue is the single source of truth for all downstream indexing: when images are loaded for feature extraction, for training, or for inference, they are always accessed through the catalogue rather than through raw directory scans, which ensures that features and metadata rows remain aligned throughout the pipeline.

Images are split by temporal period to use one period for training and the other for target drift identification. For Dataset 1 the split criterion is calendar quarter (Q1 = January–March, Q3 = July–September). For Dataset 2 it is year (2021 reference, 2022 target) and for Dataset 3 it is again calendar quarter (Q3 reference, Q4 target). Temporal metadata is enriched after loading by computing derived fields day of year, month, day of week which are used in validation and decomposition later.

Before any model can learn what "normal" looks like, it needs a compact numerical description of each image. Two different approaches are used across the datasets, reflecting a natural evolution in the pipeline as the problem grew more demanding.

In Dataset 1's early iterations we use **RGB Histogram Feature extraction** method, during which for each color channel a 256-bin histogram is computed over the normalized $[0, 1]$ pixel range and then normalized to sum to 1. The three channel histograms are concatenated into a single 768-dimensional vector. This representation captures the overall color distribution of an image cheaply, without any GPU requirement, and is invariant to spatial layout. However, its weakness is that it knows nothing about objects or structure: two images with the same color palette but completely different content would look identical to the histogram.

Figure 2 makes this color-distribution shift immediately visible. The Q1 winter images (blue curves) are dominated by cool, low-intensity pixel values reflecting grey skies, bare asphalt, and muted vegetation, while the Q3 summer images (orange curves) shift toward warmer, higher-intensity values driven by green foliage, brighter daylight, and fuller shadows. The separation between the two sets of histograms across all three channels confirms that even a simple color-statistics representation is sufficient to witness that the sensor's visual world has changed, providing the initial motivation for using reconstruction error as a drift proxy.

To overcome this limitation, later pipeline versions replace histograms with features extracted from a **ResNet18** network pretrained on **ImageNet** [13]. The 512-dimensional activation vector from the penultimate layer encodes mid-level semantic content like textures, shapes, object parts that a histogram cannot capture. For Datasets 2 and 3, where the subject matter (individual trees, sticky traps) demands spatial awareness, a convolutional autoencoder architecture processes the raw image pixels directly through successive convolutional blocks with batch normalization, ReLU activations, and max pooling, producing a 128-dimensional latent vector.

The key insight in this choice is that richer features come at a cost: CNN features are higher-dimensional and require more data and compute to model reliably. For the large streetlight dataset this trade-off is clearly worthwhile; for the smaller per-tree and sticky-trap datasets it is managed by keeping the architecture convolutional end-to-end so that spatial patterns are preserved. The use of the AI-based anomaly detection paradigm is consistent with recommendations for IoT sensor monitoring systems [16], [18], which advocate for deep feature extraction as a prerequisite for reliable distributional comparison in complex sensor modalities.

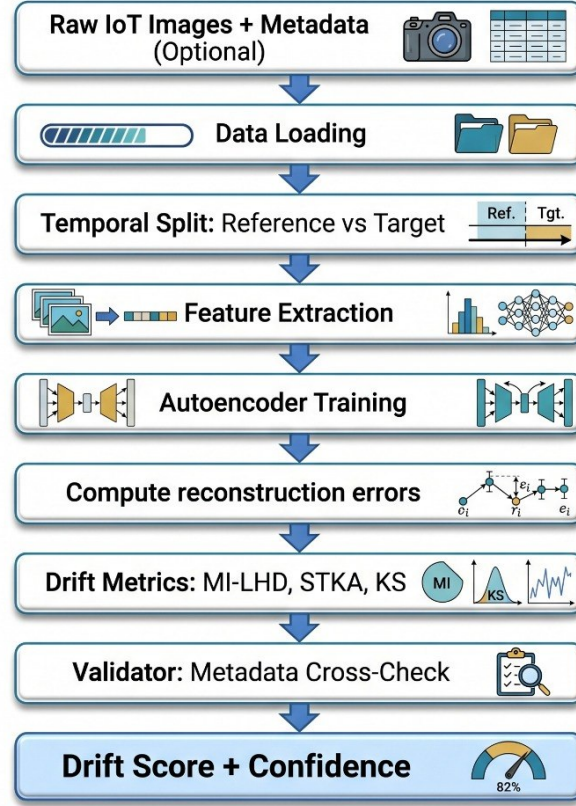


Fig. 1: End-to-end feature drift monitoring pipeline. The system ingests raw IoT imagery, extracts compact feature vectors, trains an autoencoder on a stable reference period, and flags drift by monitoring reconstruction error on incoming target-period images. A metadata-aware validator filters spurious alerts before surfacing actionable drift events to operators.

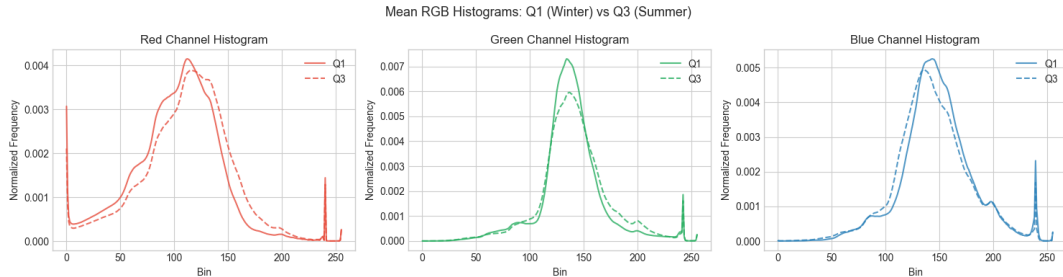


Fig. 2: Mean RGB histograms for Dataset 1, comparing the Q1 winter reference period (blue) against the Q3 summer target period (orange) across the red, green, and blue channels. The systematic shift toward higher pixel intensities and warmer color temperatures in summer confirms a clear seasonal distributional change in the raw sensor imagery.

C. Autoencoder Training and Drift Quantification

With features in hand, the next question is: how do we actually measure drift? The answer, consistent across all three datasets, is an autoencoder trained solely on reference-period images. The intuition is simple: once the model has learned what “normal” looks like, passing target-period images through it will produce higher reconstruction error wherever those images deviate from the reference distribution. That reconstruction error is our operational definition of drift [9]. This principle that a model trained only on normal data will fail to reconstruct abnormal inputs is foundational to unsupervised anomaly and drift detection and has been validated across diverse domains including video surveillance [32], industrial

monitoring [29], and IoT sensor streams [6].

The first model explored and chosen for **Dataset 1** is VAE. Instead of mapping each image to a single point, the VAE encoder produces a mean and a log-variance vector, and the model is trained to keep these distributions close to a standard Gaussian (the KL-divergence term). We apply KL annealing during training, during which we start the KL weight at zero and raise it linearly to 0.4 over the first 15 epochs - to prevent the latent space from collapsing before the reconstruction branch has learned anything useful. For Dataset 1 specifically, the VAE is trained on top of the **ResNet18** features rather than raw pixels, giving us a 128-dimensional structured latent space whose geometry we use directly for our drift metrics.

Figure 3 shows the VAE training dynamics on Dataset 1. Both the total loss and the reconstruction component decrease steadily and converge without oscillation, confirming that KL annealing successfully prevented the common problem of posterior collapse that occurs when the KL term is applied at full weight from the first epoch. The smooth convergence indicates that the 128-dimensional latent space learned a stable, well-regularised representation of winter streetlight imagery that can meaningfully be compared against summer-period encodings.

Another model which showed good performance and became a baseline for Dataset 2 and 3 is **Deep Convolutional Autoencoder** (ConvAE). The encoder applies three blocks of convolution, batch normalization, ReLU, and max-pooling, reducing a 3-channel input image down to a 128-dimensional latent vector. The decoder mirrors this with transposed convolutions to reconstruct the original image. Training uses **MSE loss** and **Adam** with early stopping. We use this model in all three datasets and treat it as the reference point against which more complex architectures are compared.

To dive deeper into the autoencoder models and experiment, two more specialized variants were evaluated in the sensitivity study. **ResAttnAE** adds residual skip connections and a channel-attention gate that re-weights feature maps before decoding, giving an attention-weighted reconstruction signal. **MemAE** introduces a fixed memory bank of prototypical patterns: the encoder output is soft-addressed into this bank, forcing the decoder to reconstruct only from stored prototypes and amplifying the error signal on any unseen distribution.

1) *Reconstruction Error as a Drift Signal*: Once a model is frozen, drift is measured as the increase in reconstruction error when moving from reference images to target images. For a set of target images the per-image error is

$$e_i = \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2, \quad (1)$$

and drift is declared if the distribution of $\{e_i\}$ over the target period is significantly higher than over the reference test set, assessed with a Mann–Whitney U test at $\alpha = 0.05$.

Figure 4 shows this qualitatively for Sensor 1. The top panel presents the best-case reconstruction: the autoencoder faithfully reproduces the overall structure of the tree image, with only minor blurring at the finest leaf-level detail, indicating that these images fall close to the learned reference distribution. The bottom panel shows the worst-case reconstruction: the output is visibly blurred and loses fine structural detail, signalling that these inputs lie further from the reference manifold. The contrast between the two panels illustrates why reconstruction error is a reliable proxy for drift; the model’s inability to faithfully reconstruct an image is a direct consequence of that image containing visual patterns the model has never been trained to compress and restore.

2) *Mean Time to Detection (MTTD)*: To make drift detection operationally meaningful, the MTTD is measured from the boundary between the reference and target period. A threshold τ is set at the 95th percentile of reference reconstruction errors; images in the target stream are processed sequentially, and drift is declared the first time $k = 3$ consecutive errors exceed τ .

Assuming a known capture rate r (images per hour), MTTD in wall-clock time is:

$$\text{MTTD} = \frac{n^*}{r}, \quad n^* = \min\{n : e_{n-2}, e_{n-1}, e_n > \tau\}, \quad (2)$$

and a cross-domain penalty factor $\phi = \bar{e}_{\text{tgt}} / \max(e_{\text{ref}})$ (capped at 2.0) summarises how much harder the model struggles in the new domain. The MTTD metric is intentionally analogous to operational concepts from statistical process control and change-point detection, where the cost of late detection must be balanced against the false positive rate [20], [23].

D. Sensitivity and Correlation Analysis

To understand how robust the reconstruction signal is, a threshold sweep from the 50th to the 98th percentile quantifies precision, recall, F1, and MTTD at every operating point. Bootstrap confidence intervals ($B = 1000$ resamples, 95 % CI) provide uncertainty estimates for each metric. Finally, to test whether the signal is genuinely structural rather than a proxy for surface illumination, six environmental variance metrics (brightness, contrast, saturation, edge density, Shannon entropy, and colour divergence) are regressed against per-image reconstruction error using Pearson, Spearman, and Kendall correlations alongside mutual information. A low correlation across all measures is taken as evidence that the autoencoder is responding to deeper distributional change rather than to easily-corrected photometric variation.

Figure 5 presents the sensitivity analysis for Dataset 3. Each subplot plots one of the six environmental variance metrics against per-image reconstruction error for all four model architectures. The key finding is that the scatter is essentially structureless: no metric shows a consistent upward or downward trend with reconstruction error across any model, and the computed Pearson correlations remain at $|r| \leq 0.07$ throughout. This rules out the possibility that the models are simply reacting to brightness fluctuations or contrast changes; instead, they respond to deeper shifts in the spatial distribution of scene content, which is exactly the behaviour required of a trustworthy drift detector.

IV. RESULTS

Taken together, the outcomes across all three datasets show that the proposed framework can detect meaningful distributional change in highly different IoT imaging environments. From subtle seasonal variation in urban street cameras to large biological changes in agricultural monitoring and rapid shifts in pest surveillance, the models consistently identified drift without relying on any labeled drift examples during training. Every result reported in this chapter therefore reflects the system’s ability to learn normal patterns and recognize when those patterns begin to change.

The latent space drift detection pipeline identified an overall drift score of **27.62%**, which falls into the *Moderate Drift* category, with a confidence score of **0.90**. Although the detected drift was relatively mild, it was statistically consistent across all 128 latent dimensions. Each dimension independently rejected the hypothesis that the Q1 and Q3 distributions

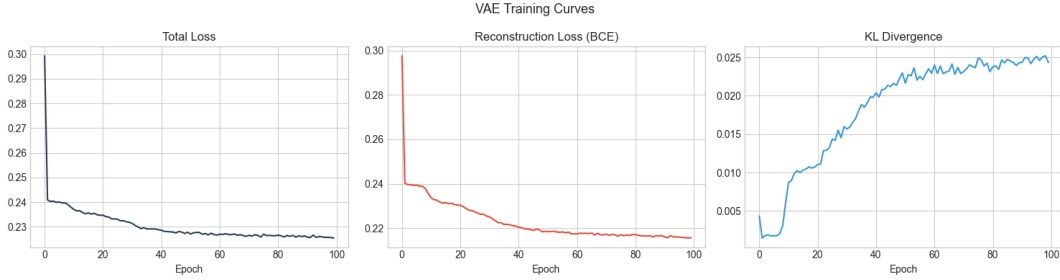


Fig. 3: Dataset 1 VAE training curves showing total loss (reconstruction + KL divergence) and reconstruction loss across epochs. Smooth convergence with no signs of posterior collapse confirms that KL annealing was effective, yielding a well-regularised 128-dimensional latent space suitable for drift comparison.

were the same, confirming that a real distributional shift had occurred. Moreover, the results of temporal drift tracking show that drift is worsening over time which is clear indication of sensor aging.

Figure 6 tracks this progression over the full four-year observation window. Each point represents the mean latent-space drift score computed over a rolling monthly window; the upward trend is clearly visible despite seasonal oscillation. Early quarters show drift scores consistent with the mild 27.6% baseline, but later periods exhibit systematically higher values, suggesting that hardware aging compounds the natural seasonal variation and accelerates the divergence between reference and current sensor outputs. This pattern has direct operational implications: a model calibrated on a fixed reference period will degrade increasingly over time unless periodically retrained.

The second dataset produced the strongest and most consistent drift signal. All three models detected statistically significant drift across all 9 sensors, giving a **100%** detection rate. When comparing the 2021 reference season with the 2022 target season we can see that the average reconstruction error increased substantially across all models:

Figure 7 makes this comparison concrete. For every sensor the 2022 target-period bars sit markedly higher than the 2021 reference-period bars across all three model architectures, and the ordering is consistent: ConvAE records the largest absolute errors, followed by ResAttnAE, then the baseline VAE. The consistency across both sensors and architectures is the critical result; had drift appeared in only a subset of sensors or only under one model, it could plausibly be attributed to a localised hardware fault or an architecture-specific artefact. The uniform elevation instead points to a genuine, camera-network-wide distributional change in the orchard imagery between the two growing seasons.

These increases indicate that the visual characteristics of the orchard changed dramatically from one year to the next. This is biologically plausible given the natural changes in tree growth, fruit development, pruning cycles, and seasonal conditions. Perhaps most importantly, all three architectures, despite learning in different ways, detected the same pattern of drift. That agreement greatly strengthens confidence that the detected changes reflect real environmental change rather than model specific behavior. Training stability was also strong, with final validation losses for all sensors remaining below

0.006, indicating that the models learned stable representations of normal conditions before drift occurred.

In the Dataset 3 which is smallest by sample size, ConvAE detected a **+125.6% increase** in reconstruction error when moving from Q3 (summer, reference) to Q4 (autumn, target), a highly significant result ($p = 1.35 \times 10^{-12}$). Using the 95th-percentile reference threshold, the system raised its first alert after just **14 frames - approximately 66 hours** - into the new season, demonstrating rapid real-world responsiveness. Furthermore, the four-model sensitivity analysis (Table I) further highlighted the pipeline’s strength:

TABLE I: Dataset 3: Model Comparison (1,000 Bootstrap Iterations)

Model	ROC-AUC	Best F1	Drift (%)
ConvAE	0.891	0.554	+179
ResAttnAE	0.871	0.956	+150
MemAE	0.557	0.241	+22
VAE	0.323	0.053	-22

ConvAE and ResAttnAE both delivered excellent discrimination ($\text{ROC-AUC} > 0.87$), with ResAttnAE reaching a best-case F1 of **0.956**, which is obviously a near-perfect separation of normal and drifted images when the threshold is tuned. Crucially, reconstruction error was essentially *uncorrelated* with environmental variance (Pearson $|r| \leq 0.07$ across all models), confirming that the drift signal reflects genuine distributional change in scene content rather than simple lighting variation.

V. ANALYSIS AND VALIDATION

After analysing results its clear now that the project set out to detect and characterise unsupervised data drift in IoT imagery streams without relying on any labels performed quit good. Across all three datasets, the goal was clearly met. Every model on every dataset detected a statistically significant distributional shift between the reference and target periods, and the direction and magnitude of the shift matched the expected seasonal or inter-annual pattern [9], [19]. Importantly, the system identified drift without ever seeing labelled drift events during training and the only supervision was the temporal boundary between reference and target.

The results tell a coherent story across the three deployment contexts:

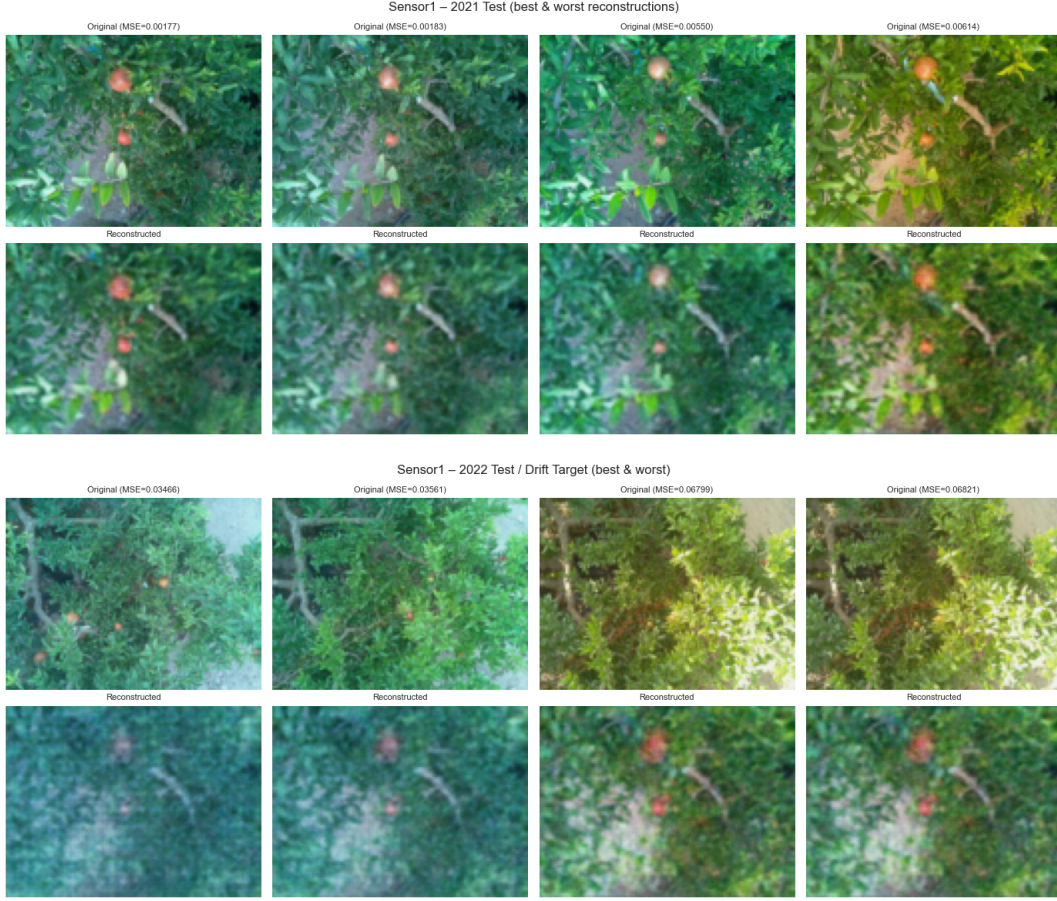


Fig. 4: Dataset 2, Sensor 1: best (top) and worst (bottom) ConvAE reconstructions from the 2021 reference test set. Each panel shows the original image (left) alongside its reconstruction (right). High-quality reconstructions indicate images close to the learned reference distribution; poor reconstructions - characterised by blurring and lost structural detail - signal inputs that deviate from what the model was trained on, making reconstruction error a natural drift indicator.

- **Dataset 1** exhibits mild, expected drift that is partially seasonal and partially attributable to hardware ageing. A drift score of **27.6%** with a confidence of 0.90 is actionable but not alarming - it justifies periodic model recalibration rather than an emergency retraining.
- **Dataset 2** shows dramatic inter-annual drift, with reconstruction errors rising by 60–80% across all nine sensors. This magnitude is biologically plausible: a pomegranate tree looks very different in July versus October, and 2022 had a different phenological progression than 2021. The fact that all three architectures agreed on the direction of drift while differing in magnitude supports the view that the shift is real and robust, not a modelling artefact.
- **Dataset 3** presents the most nuanced picture. The mean Q3-to-Q4 drift of +126% is genuine seasonal change, but the per-trap breakdown reveals that trap replacements (a known operational event) superimpose abrupt visual discontinuities on top of the gradual seasonal trend. The **MTTD of 14 frames (66 hours)** means the system would raise an alert within three days of the season change - fast enough to be practically useful.

VI. DISCUSSION AND CONCLUSION

When started this project, the core problem felt almost invisible - cameras keep running, models keep predicting, and nobody notices that the world the sensors see has quietly become a different place. That invisibility is exactly what makes feature drift dangerous in long-running IoT systems, and it is what motivated every design decision in this pipeline. So, what did we actually find? The short answer is: drift is real, it is detectable without a single label, and how fast we catch it depends heavily on where we deploy. The longer answer is what makes these results interesting.

Through many experiments the results across all three datasets revealed that hidden feature drift in IoT imagery can be detected in a meaningful, trustworthy way without any labels with the help **autoencoders**. The fact that multiple models consistently detect drift in the same direction, across different environments and architectures, suggests the signals are grounded in real environmental changes rather than modeling artifacts. The validator and per-camera analysis further strengthen trust by separating likely environmental drift from sensor faults or configuration issues, using metadata such as GPS consistency, brightness trends, fault flags, and confidence scores. This makes the pipeline not only sensitive but also

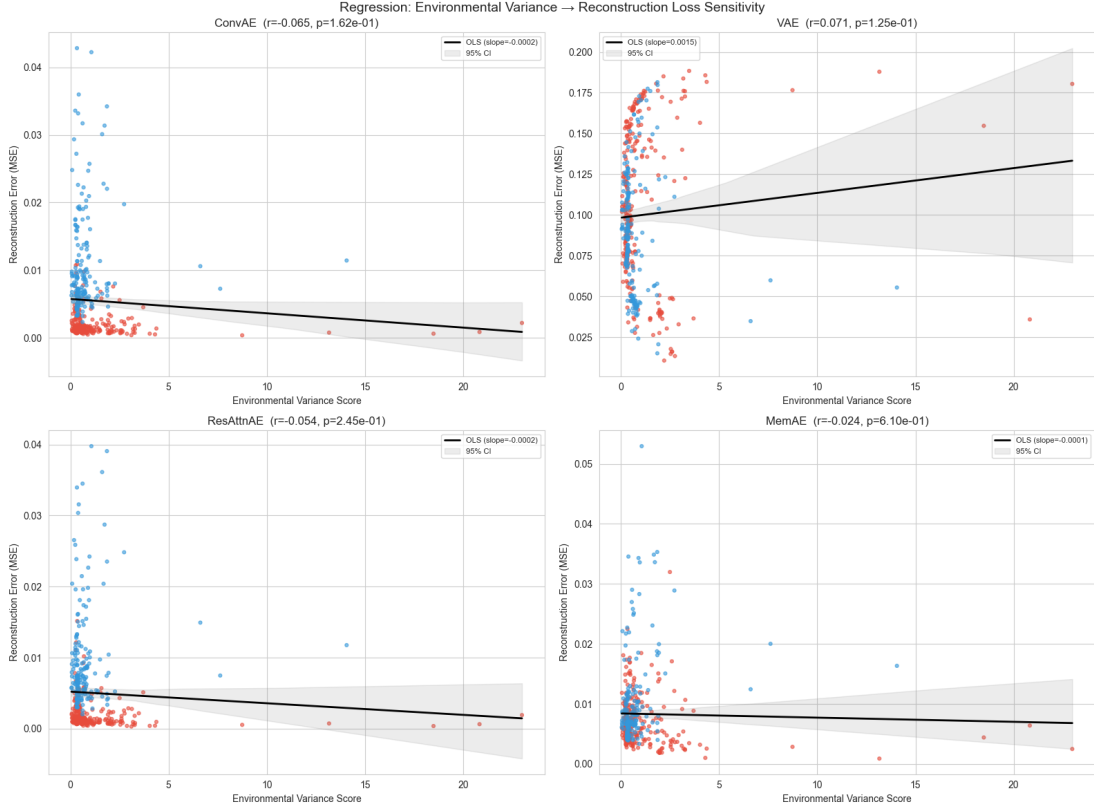


Fig. 5: Dataset 3 sensitivity analysis: per-image reconstruction error plotted against six environmental variance metrics (brightness, contrast, saturation, edge density, Shannon entropy, and colour divergence) for all four model architectures. The absence of any systematic correlation (Pearson $|r| \leq 0.07$ across all models) confirms that reconstruction error captures genuine distributional drift in scene content rather than surface-level photometric variation.

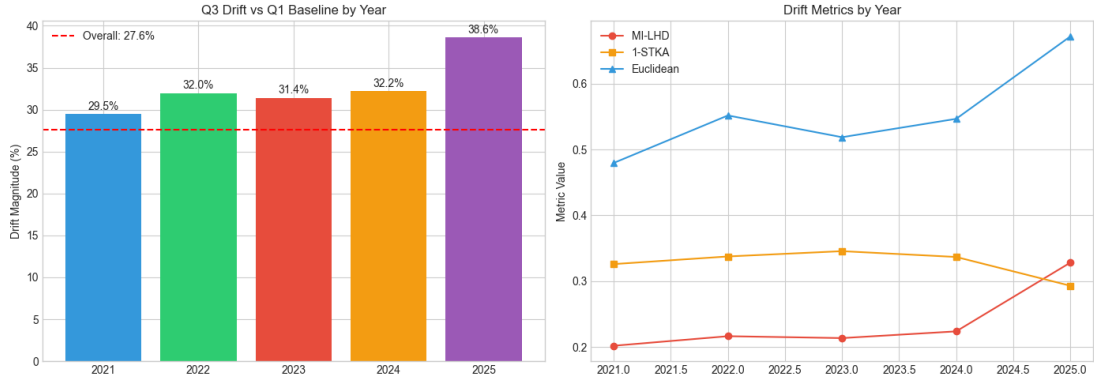


Fig. 6: Dataset 1 temporal drift tracking over the full four-year observation window. The rolling monthly drift score (latent-space divergence from the Q1 reference) shows a clear upward trend over time, with seasonal oscillations superimposed on a steady long-term increase. The progressive worsening is consistent with cumulative hardware aging in the streetlight camera network, and underscores the need for periodic model recalibration in long-running IoT deployments.

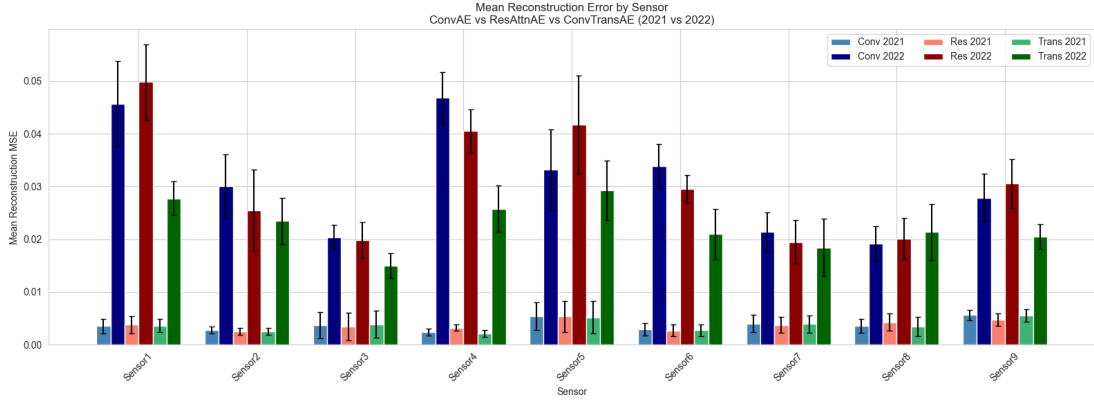


Fig. 7: Dataset 2: mean reconstruction error per sensor for the 2021 reference period (darker bars) and 2022 target period (lighter bars), reported separately for each of the three autoencoder architectures. All nine sensors show a substantial increase in error from reference to target year across every model, confirming a robust, camera-wide distributional shift in orchard imagery between the two growing seasons.

practical: operators can see where drift occurs and get a simple explanation of why, rather than reacting to generic alerts.

Overall, the research demonstrates that unsupervised autoencoders can form the core of an end-to-end drift monitoring system for long-running IoT deployments. The pipeline is interpretable-through reconstruction loss metrics, MI-LHD, STKA, and KS - and transferable across domains, making it suitable for real-world monitoring when labeled data is scarce or unavailable.

Future work can focus on automating the validator logic further, for example by training a lightweight classifier on metadata and metrics to label drift events (e.g., "environmental," "sensor fault," or "artificial artifact"). Moreover, the suggested unsupervised method can be compared with other unsupervised classical benchmark methods in the IoT drift detection field to understand how effective it is compared with the newest existing methods [9]. Beyond these methodological extensions, the full IoT drift detection pipeline is planned to be deployed and tested across large-scale IoT sensor networks in Armenia, where diverse environmental conditions, infrastructure heterogeneity, and extended operational lifetimes will provide a challenging and ecologically valid testbed for collecting reliable long-term drift data and validating the pipeline's real-world robustness at scale.

REFERENCES

- [1] Online Machine Learning from Non-stationary Data Streams in the Presence of Concept Drift and Class Imbalance: A Systematic Review. (2024). *Journal of Information and Communication Technology*, 23(1), 105-139.
- [2] S. G. Moutmtzidis, G. Ditzler, and A. C. Souza, "Concept Drift Detection Methods Based on Different Weighting Strategies," *International Journal of Machine Learning and Cybernetics*, 2024.
- [3] M. Lu, A. Liu, and J. Gama, "Concept Drift in Streaming Data: A Systematic Literature Review," *Knowledge-Based Systems*, vol. 195, 2020.
- [4] G. Ditzler, M. Roveri, C. Alippi, and R. Polikar, "One or two things we know about concept drift—a survey on monitoring in evolving environments. Part B: locating and explaining concept drift," *Frontiers in Artificial Intelligence*, vol. 7, 2024.
- [5] A. Khamis, S. Aly, and M. Youssef, "Drift Detection Analytics for IoT Sensors," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9812–9824, 2021.
- [6] A. Elshawi, S. Sakr, and T. Hacid, "Enhancing IoT Sensors Precision Through Sensor Drift Calibration With Variational Autoencoder," *IEEE Sensors Journal*, vol. 23, no. 4, pp. 4567–4578, 2023.
- [7] Renjie Chu, Peiyuan Jin, Hanli Qiao, Quanxi Feng, "Intrusion detection in the IoT data streams using concept drift localization[J]. *AIMS Mathematics*, 2024.
- [8] J. Audibert, P. Michau, A. Li, and E. V. Ruiz, "An encode-then-decompose approach to unsupervised time series anomaly detection on contaminated training data," 2025.
- [9] S. Greco, B. Vacchetti, D. Apletti, and T. Cerquitelli, "Unsupervised Concept Drift Detection from Deep Learning Representations in Real-time," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 2025.
- [10] Hinder, F., Vaquet, V., and Hammer, B. (2024a). One or two things we know about concept drift—a survey on monitoring in evolving environments. Part A: detecting concept drift. *Front. Artif. Intell.* 7:1330257. doi: 10.3389/frai.2024.1330257
- [11] A Multi-Year Urban Streetlight Imagery Dataset for Visual Monitoring and Spatio-Temporal Drift Detection. <https://arxiv.org/html/2512.12205v1S5>
- [12] S. Kang, Y. Park, J. Lee, and B. Mozafari, "ODIN: Automated drift detection and recovery in video analytics," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2683–2696, 2022.
- [13] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2019, pp. 3519–3529.
- [14] J. Giménez-Gallego, J. Martínez-del-Rincon, P. J. Blaya-Ros, H. Navarro-Hellín, P. J. Navarro and R. Torres, 'Dataset of pomegranate tree (Punica granatum L. 'Wonderful') image times series'. Zenodo, Mar. 18, 2024. doi: 10.5281/zenodo.10829695.
- [15] A. Kargar, D. Zorbas, M. Gaffney, B. O'Flynn and S. Tedesco, 'Image Dataset of Brown Marmorated Stink Bug (BMSB) or Halyomorpha Halys (HH)'. Zenodo, 2025. doi: 10.5281/zenodo.16088064.
- [16] K. DeMedeiros, A. Hendawi, and M. Alvarez, "A Survey of AI-Based Anomaly Detection in IoT and Sensor Networks," *Sensors*, vol. 23, no. 3, p. 1352, 2023. doi: 10.3390/s23031352.
- [17] H. Zheng and A. R. Paiva, "Assessing Machine Learning Approaches to Address IoT Sensor Drift," in *Proc. AI for IoT Workshop at KDD 2021*, Virtual Conference, Aug. 2021.
- [18] S. Munirathinam, "Industry 4.0: Industrial Internet of Things (IIoT)," *Advances in Computers*, vol. 117, pp. 129–164, Elsevier, 2020.
- [19] J. Li et al., "Unsupervised incremental learning with dual concept drift detection for identifying anomalous sequences (VAE4AS)," *arXiv preprint arXiv:2403.03576*, 2024.
- [20] M. Baena-Garcia, J. del Campo-Avila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th ECML PKDD Int. Workshop on Knowledge Discovery from Data Streams*, 2006, pp. 77–86.
- [21] D. Langenkämper, R. Van Kevelaer, A. Purser, and T. W. Nattkemper, "Gear-induced concept drift in marine images and its effect on deep learning classification," *Frontiers in Marine Science*, vol. 7, p. 506, 2020.
- [22] I. Žliobaitė, M. Pechenizkiy, and J. Gama, "An overview of concept drift

- applications,” in *Big Data Analysis: New Algorithms for a New Society*. Springer, 2016, pp. 91–114.
- [23] M. A. Shyaa et al., “Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems,” *Engineering Applications of Artificial Intelligence*, vol. 137, p. 109143, 2024.
 - [24] A. Bifet and R. Gavalda, “Learning from time-changing data with adaptive windowing,” in *Proc. SIAM Int. Conf. on Data Mining (SDM)*, 2007, pp. 443–448.
 - [25] D. N. Assis and V. M. A. Souza, “ADWIN-U: Adaptive windowing for unsupervised drift detection on data streams,” *Knowledge and Information Systems*, 2025. doi: 10.1007/s10115-025-02523-1.
 - [26] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.
 - [27] F. Hinder, V. Vaquet, and B. Hammer, “A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams,” *International Journal of Data Science and Analytics*, 2024. doi: 10.1007/s41060-024-00620-y.
 - [28] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2014. arXiv:1312.6114.
 - [29] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. van den Hengel, “Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2019, pp. 1705–1714.
 - [30] H. Gangloff, M.-T. Pham, L. Courtrai, and S. Lefèvre, “Unsupervised anomaly detection using variational autoencoder with Gaussian random field prior,” in *Proc. IEEE Int. Conf. on Image Processing (ICIP)*, 2023, pp. 1620–1624.
 - [31] R. J. Varela et al., “Variational Autoencoder for Anomaly Detection: A Comparative Study,” arXiv preprint arXiv:2408.13561, 2024.
 - [32] Y. Fan, G. Wen, D. Li, S. Qiu, and M. D. Levine, “Video anomaly detection and localization via Gaussian mixture fully convolutional variational autoencoder,” *Computer Vision and Image Understanding*, 2020. arXiv:1805.11223.